

UV/PIP Installation - Why Packaging Matters

Adding *setup.py* makes the project installable as a proper Python package. This ensures import paths behave consistently whether running locally, in tests, or in CI. Editable installs (`pip install -e .`) are useful during development since code changes take effect immediately. *uv* can also read *install_requires* from *setup.py* when syncing environments, making it a central location for core dependencies.

SQL Injection Defenses

All SQL queries were refactored away from using raw string construction. This is a better solution because without fstring or concatenation, user input never interacts with SQL text directly. *pg_sql.SQL()* wraps every query statement, which separates construction from execution and this pattern is copied for every query.

LIMIT Enforcement

Every query in *query_data.py* enforces a LIMIT clause to prevent unbounded data exposure. A *clamp_limit* function ensures user-supplied values are always within the range 1–100. This ensures that no endpoint can return unbounded data regardless of user input.

Least-Privilege DB Configuration

I created a dedicated PostgreSQL user *gradcafe_reader* with SELECT-only access on the *applicants* table. The permissions granted are shown below:

```
CREATE USER gradcafe_reader WITH PASSWORD ...;
```

```
GRANT CONNECT ON DATABASE applicantdata TO gradcafe_reader;
```

```
GRANT USAGE ON SCHEMA public TO gradcafe_reader;
```

```
GRANT SELECT ON TABLE applicants TO gradcafe_reader;
```

This user cannot INSERT, UPDATE, DELETE, DROP, or ALTER any data or schema. Since the app is read-only, no write permissions are needed or granted. Credentials are loaded from system environment variables.

```
applicantdata=# CREATE USER gradcafe_reader WITH PASSWORD
CREATE ROLE
applicantdata=# GRANT CONNECT ON DATABASE applicantdata TO gradcafe_reader;
GRANT
applicantdata=# GRANT USAGE ON SCHEMA public TO gradcafe_reader;
GRANT
applicantdata=# GRANT SELECT ON TABLE applicants TO gradcafe_reader;
GRANT
applicantdata=# du\
invalid command \
Try \? for help.
applicantdata=# \du

               List of roles
-----
Role name | Attributes
-----
gradcafe_reader |
postgres | Superuser, Create role, Create DB, Replication, Bypass RLS

applicantdata=# \dp applicants

Schema | Name | Type | Access privileges | Column privileges | Policies
-----+-----+-----+-----+-----+-----
public | applicants | table | postgres=arwdDxtm/postgres+ |  | 
      | | | gradcafe_reader=r/postgres |  | 
(1 row)
```

Dependency Graph

On the left side of the dependency graph, *src.app* sits at the center of the Flask cluster, which is dependent on flask and its internal modules, as well as *dotenv* for loading environment variables. On the right side, *src.load_data* and *src.query_data* both depend heavily on *psycopg* and its submodules, including *psycopg.sql* which is used for safe SQL composition to prevent injection attacks. The *psycopg* library manages a lot of dependencies including connection management and cursor handling. The configuration module does not appear in the graph because it only relies on Python's standard library (*os* and *json*). In general terms, *Flask* handles routing and HTTP, *psycopg* handles database access, and *dotenv* handles secure credential loading.

SNYK Analysis

Snyk identified 1 high-severity vulnerability in *diskcache@5.6.3* — a deserialization of untrusted data issue (SNYK-PYTHON-DISKCACHE-15268422). This package is a dependency that was introduced through the LLM/scrapper pipeline; it is not directly a dependency of the Flask app. No upgrade or patch is currently available from Snyk. Since *diskcache* is only used by the scraper's LLM component, this is a low risk to the web app.

```
Ⓢ $ snyk test --file=requirements.txt

Testing C:\Users\stefa\JHU\ModernSoftwareConcepts_Summer26\jhu_software_concepts\module_5...

Tested 70 dependencies for known issues, found 1 issue, 2 vulnerable paths.

Issues with no direct upgrade or patch:
  X Deserialization of Untrusted Data [High Severity][https://security.snyk.io/vuln/SNYK-PYTHON-DISKCACHE-15268422] in diskcache@5.6.3
    introduced by diskcache@5.6.3 and 1 other path(s)
    No upgrade or patch available

Organization: stefan-thomas26
Package manager: pip
Target file: requirements.txt
Project name: module_5
Open source: no
Project path: C:\Users\stefa\JHU\ModernSoftwareConcepts_Summer26\jhu_software_concepts\module_5
Licenses: enabled

Tip: Detected multiple supported manifests (2), use --all-projects to scan all of them at once.
```

Snyk Code Scan (SAST)

Snyk Code (SAST) identified 7 issues across the codebase, all of which are of low severity. They are all Path Traversal warnings in the scraper pipeline (run_web_scraper.py, llm_hosting/app.py) where file paths flow from command-line arguments or environment variables into file open/write operations.

```
$ snyk code test
Open Issues

X [LOW] Path Traversal
Finding ID: 644b2eee-04a0-4865-bf8f-da01c8151a55
Path: src/load_data.py, line 157
Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: 99cc788b-463e-4392-9aea-843eb09988af
Path: src/web_scraper/llm_hosting/app.py, line 319
Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: e63e49b2-0648-4e66-b2b6-e32e895b566c
Path: src/web_scraper/run_web_scraper.py, line 208
Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: 62bd373c-f977-4b9a-82c9-27bd311eec23
Path: src/web_scraper/llm_hosting/app.py, line 327
Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

X [LOW] Path Traversal
Finding ID: bf6ae26e-74ec-4a4b-8806-8de980b91fd9
Path: src/web_scraper/llm_hosting/app.py, line 340
Info: Unsanitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

X [LOW] Path Traversal
Finding ID: d923d0af-39e6-4a7d-b27c-6532f831afda
Path: src/web_scraper/run_web_scraper.py, line 138
Info: Unsanitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

X [LOW] Path Traversal
Finding ID: ce6fd7ab-6b3e-4802-84a2-13077f21fe3c
Path: src/web_scraper/run_web_scraper.py, line 190
Info: Unsanitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

Test Summary

Organization: stefan-thomas26
Test type: Static code analysis
Project path: C:\Users\stefa\JHU\ModernSoftwareConcepts_Summer26\jhu_software_concepts\module_5

Total issues: 7
Ignored issues: 0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues: 7 [ 0 HIGH 0 MEDIUM 7 LOW ]

Tip

To view ignored issues, use the --include-ignores option.
```

CI Workflow

The GitHub Actions workflow at *.github/workflows/ci.yml* runs on every push and pull request to master:

Job 1 — Pylint: Runs `pylint src/ --fail-under=10` and fails the pipeline if any file scores below 10/10.

Job 2 — Dependency Graph: Installs Graphviz and pydeps, generates `dependency.svg`, and fails if the file is not produced. The graph is saved as an artifact.

Job 3 — Snyk: Installs the Snyk CLI and runs `snyk test --file=requirements.txt` using a `SNYK_TOKEN` secret. This scans all dependencies for known vulnerabilities on every push.

Job 4 — Pytest: Spins up a PostgreSQL 15 service container, installs all dependencies, and runs the full test suite. Coverage must be 100% for the pipeline to pass.